

1. Задачи к выполнению

1.1. Часть 1. Абстрактные типы данных: очередь

Очереди обычно упорядочивают элементы по принципу FIFO («первым пришёл — первым ушёл»): все новые элементы добавляются в конец очереди, а первым удаляется методами `remove()` / `poll()` тот элемент, который был добавлен раньше остальных.

1.1.1. Задача 1

Реализуйте стек (LIFO — «последний пришёл — первый ушёл»), используя не более двух очередей (FIFO — «первый пришёл — первый ушёл») и только стандартные методы очереди `add` / `offer`, `peek`, `poll`, `size`, `isEmpty`. Для проектирования используйте отношение композиция.

В реализованном стеке должны поддерживаться методы `push`, `top`, `pop`, `empty` и метод, возвращающий строковое представление всех элементов стека.

Методы класса `StackOnQueue`:

- `void push(int x)` — помещает элемент `x` на вершину стека.
- `int pop()` — удаляет элемент на вершине стека и возвращает его.
- `int top()` — возвращает элемент на вершине стека.
- `boolean empty()` — возвращает `true`, если стек пуст, в ином случае `false`.

1.1.1.1. Решение

Идея реализации: два поля `queue1` / `queue2` типа `Queue<Integer>` (отношение композиции). При каждом `push(x)` новый элемент добавляется в пустую `queue2`, после чего в неё же переносятся все элементы из `queue1`, и ссылки на очереди меняются местами. В результате голова `queue1` всегда содержит последний добавленный элемент, что и даёт поведение LIFO при использовании только операций FIFO-очереди.

```
1 // queue/StackOnQueue.java
2 package ru.haykor.mirea.queue;
3
4 import java.util.LinkedList;
5 import java.util.Queue;
6
7 public class StackOnQueue {
8     private Queue<Integer> queue1 = new LinkedList<>();
9     private Queue<Integer> queue2 = new LinkedList<>();
```

```

10
11     public void push(int x) {
12         queue2.offer(x);
13         while (!queue1.isEmpty()) {
14             queue2.offer(queue1.poll());
15         }
16
17         Queue<Integer> temp = queue1;
18         queue1 = queue2;
19         queue2 = temp;
20     }
21
22     public int pop() {
23         return queue1.poll();
24     }
25
26     public int top() {
27         return queue1.peek();
28     }
29
30     public boolean empty() {
31         return queue1.isEmpty();
32     }
33
34     @Override
35     public String toString() {
36         return "стек: " + queue1.toString();
37     }
38 }

```

1.1.2. Задача 2

Создайте новый тестовый класс. Для проверки работы созданного стека создайте в тестовом классе новый экземпляр класса `StackOnQueue`, добавьте в этот стек два значения, выведите объект, находящийся на вершине стека (без удаления), выведите объект, находящийся на вершине стека и удалите его, проверьте стек на пустоту и выведите информацию о всех элементах стека.

1.1.2.1. Решение

```

1 // app/TestStackOnQueue.java
2 package ru.haykor.mirea.app;
3

```

```

4 import ru.haykor.mirea.queue.StackOnQueue;
5
6 public class TestStackOnQueue {
7     public static void main(String[] args) {
8         StackOnQueue stack = new StackOnQueue();
9
10        stack.push(10);
11        stack.push(20);
12
13        System.out.println("Вершина стека: " + stack.top());
14        System.out.println("Удалённый элемент: " +
stack.pop());
15        System.out.println("Стек пуст: " + stack.empty());
16
17        stack.push(30);
18        stack.push(40);
19        System.out.println(stack);
20    }
21 }

```

Вывод программы:

```

1 Вершина стека: 20
2 Удалённый элемент: 20
3 Стек пуст: false
4 стек: [40, 30, 10]

```

1.2. Часть 2. Порождающие паттерны проектирования

Порождающие паттерны (Creational Patterns) — это группа паттернов проектирования, которая фокусируется на процессе создания объектов: они инкапсулируют логику создания объекта отдельно от его использования, упрощают создание сложных объектов и позволяют менять способ создания объектов, не затрагивая код, который эти объекты использует. К порождающим паттернам относятся Singleton, Factory Method, Abstract Factory, Builder и Prototype.

1.2.1. Практическое задание — вариант №4 (Builder)

Разработать систему для создания заказа в ресторане. Использовать паттерн Builder для построения объекта `Order`, который включает следующие параметры: основное блюдо, гарнир, напиток, десерт.

— Создать класс `Order` с соответствующими полями.

- Реализовать класс `OrderBuilder` для пошагового создания заказа.
- Продемонстрировать сборку заказов с разными конфигурациями.

Пример функционала: метод `setMainDish(String mainDish)` для выбора основного блюда, метод `setDrink(String drink)` для выбора напитка, метод `build()`, возвращающий готовый заказ.

Паттерн Builder разделяет процесс пошагового построения объекта `Order` (класс `OrderBuilder`) от самого объекта-результата (класс `Order`), позволяя собирать разные конфигурации заказа одним и тем же кодом строителя — методы `OrderBuilder` возвращают `this`, поэтому вызовы можно объединять в цепочку.

1.2.1.1. Решение

```
1 // patterns/builder/Order.java
2 package ru.haykor.mirea.patterns.builder;
3
4 public class Order {
5     private String mainDish;
6     private String sideDish;
7     private String drink;
8     private String dessert;
9
10    void setMainDish(String mainDish) {
11        this.mainDish = mainDish;
12    }
13
14    void setSideDish(String sideDish) {
15        this.sideDish = sideDish;
16    }
17
18    void setDrink(String drink) {
19        this.drink = drink;
20    }
21
22    void setDessert(String dessert) {
23        this.dessert = dessert;
24    }
25
26    public String getMainDish() {
27        return mainDish;
28    }
29 }
```

```

30     public String getSideDish() {
31         return sideDish;
32     }
33
34     public String getDrink() {
35         return drink;
36     }
37
38     public String getDessert() {
39         return dessert;
40     }
41
42     @Override
43     public String toString() {
44         return "Order{mainDish='" + mainDish + "', sideDish='"
+ sideDish
45             + "', drink='" + drink + "', dessert='" +
dessert + "'}";
46     }
47 }

```

```

1 // patterns/builder/OrderBuilder.java
2 package ru.haykor.mirea.patterns.builder;
3
4 public class OrderBuilder {
5     private final Order order = new Order();
6
7     public OrderBuilder setMainDish(String mainDish) {
8         order.setMainDish(mainDish);
9         return this;
10    }
11
12    public OrderBuilder setSideDish(String sideDish) {
13        order.setSideDish(sideDish);
14        return this;
15    }
16
17    public OrderBuilder setDrink(String drink) {
18        order.setDrink(drink);
19        return this;
20    }
21
22    public OrderBuilder setDessert(String dessert) {

```

```

23         order.setDessert(dessert);
24         return this;
25     }
26
27     public Order build() {
28         return order;
29     }
30 }

```

```

1 // app/TestOrderBuilder.java
2 package ru.haykor.mirea.app;
3
4 import ru.haykor.mirea.patterns.builder.Order;
5 import ru.haykor.mirea.patterns.builder.OrderBuilder;
6
7 public class TestOrderBuilder {
8     public static void main(String[] args) {
9         Order lunch = new OrderBuilder()
10             .setMainDish("Стейк")
11             .setSideDish("Картофель фри")
12             .setDrink("Апельсиновый сок")
13             .setDessert("Тирамису")
14             .build();
15
16         Order quickOrder = new OrderBuilder()
17             .setMainDish("Бургер")
18             .setDrink("Кола")
19             .build();
20
21         System.out.println(lunch);
22         System.out.println(quickOrder);
23     }
24 }

```

Вывод программы демонстрирует две разные конфигурации заказа, собранные одним и тем же строителем — полный обед `lunch` со всеми четырьмя параметрами и укороченный `quickOrder` без гарнира и десерта:

```

1 Order{mainDish='Стейк', sideDish='Картофель фри',
  drink='Апельсиновый сок', dessert='Тирамису'}
2 Order{mainDish='Бургер', sideDish='null', drink='Кола',
  dessert='null'}

```